

**TRƯỜNG ĐẠI HỌC CÔNG NGHỆ
ĐẠI HỌC QUỐC GIA HÀ NỘI**



**BÁO CÁO BÀI TẬP LỚN
MÔN TRÍ TUỆ NHÂN TẠO**

Xây dựng trò chơi cờ Caro với thuật toán Minimax và Alpha-Beta Pruning

Nhóm sinh viên thực hiện:

Họ và tên	Mã sinh viên
Lê Chí Anh Tuấn	23020703
Nguyễn Thúy Hằng	23020662
Nguyễn Dương Minh Anh	23020651
Hồ Thúy Hằng	23020661

Hà Nội, 2026

Đóng góp thành viên

MSV	Họ và tên	Đóng góp	Công việc
23020703	Lê Chí Anh Tuấn	25%	Phụ trách cài đặt chương trình, xây dựng các chức năng chính của trò chơi XO, triển khai thuật toán AI và hoàn thiện hệ thống.
23020662	Nguyễn Thúy Hằng	25%	Phụ trách viết báo cáo, tổng hợp nội dung lý thuyết, mô tả thuật toán và trình bày kết quả thực nghiệm.
23020651	Nguyễn Dương Minh Anh	25%	Thiết kế các trạng thái thực nghiệm, xây dựng kịch bản kiểm thử và hỗ trợ đánh giá kết quả hoạt động của AI.
23020661	Hồ Thúy Hằng	25%	Phụ trách viết báo cáo, chỉnh sửa nội dung, hoàn thiện bố cục và chuẩn hóa tài liệu báo cáo.

Bảng 1: Bảng mức độ đóng góp của các thành viên

Về việc nhóm có 4 thành viên: Do cả 4 chúng em đều là sinh viên Khoa Công nghệ thông tin nhưng đăng ký học cùng lớp với các bạn Khoa Điện tử Viễn thông, nên việc ghép nhóm với các bạn khác gặp nhiều hạn chế. Nhóm đã xin phép và nhận được sự đồng ý của thầy từ đầu kỳ để thực hiện bài tập với 4 thành viên. Chúng em kính mong thầy thông cảm và xin chân thành cảm ơn thầy ạ.

Mục lục

Đóng góp thành viên	1
1 Mô tả bài toán cờ Caro	4
2 Luật chơi và điều kiện thắng	4
3 Biểu diễn trạng thái bàn cờ	4
4 Cách sinh nước đi hợp lệ	5
5 Cách kiểm tra trạng thái kết thúc	5
6 Thuật toán Minimax	5
7 Thuật toán Alpha-Beta	6
8 Hàm đánh giá trạng thái	7
8.1 Tổng quan phương pháp	7
8.2 Thuật toán quét và trích xuất đặc trưng	8
8.3 Hệ thống trọng số	8
9 Thiết kế các trạng thái thử nghiệm	9
9.1 Thiết kế các trạng thái điển hình	9
9.2 Thiết kế các trạng thái với các độ sâu khác nhau	15
10 Bảng kết quả thực nghiệm	23
11 Nhận xét về số trạng thái đã xét và thời gian chạy	24
12 Nhận xét về ảnh hưởng của độ sâu tìm kiếm	25
12.1 Ảnh hưởng đến thời gian chạy	25

12.2	Ảnh hưởng đến kết quả nước đi	25
13	Ưu điểm và hạn chế của chương trình	26
13.1	Ưu điểm	26
13.2	Hạn chế	26
14	Tài liệu tham khảo	27

1 Mô tả bài toán cờ Caro

Bài toán được xây dựng nhằm phát triển một tác tử AI có khả năng chơi cờ Caro với người chơi. Trong chương trình, AI được xây dựng dựa trên các thuật toán tìm kiếm trong trí tuệ nhân tạo như **Minimax** và **Alpha-Beta Pruning**. Thuật toán Minimax giúp máy tính đánh giá các trạng thái của bàn cờ để lựa chọn nước đi tối ưu, trong khi Alpha-Beta Pruning hỗ trợ cắt tỉa các nhánh tìm kiếm không cần thiết nhằm giảm thời gian xử lý và tăng hiệu quả của chương trình.

Github: XOGame

2 Luật chơi và điều kiện thắng

Hai người chơi luân phiên đặt quân **X** và **O** vào một ô trống hợp lệ trên bàn cờ 9×9 . Chương trình hỗ trợ cả chế độ hai người chơi lẫn chế độ đấu với AI. Một ván đấu kết thúc khi:

- Một bên tạo được 4 quân liên tiếp theo hàng ngang, hàng dọc hoặc đường chéo (kể cả khi bị chặn hai đầu) thì bên đó thắng.
- Bàn cờ đầy mà không ai đạt điều kiện trên thì kết quả là hòa.

3 Biểu diễn trạng thái bàn cờ

Trạng thái bàn cờ được biểu diễn bằng ma trận hai chiều 9×9 , trong đó mỗi ô nhận một trong ba giá trị: ô trống, quân X, hoặc quân O. Trạng thái ban đầu là ma trận toàn ô trống; sau mỗi nước đi, ô tương ứng được cập nhật theo người vừa đi.

4 Cách sinh nước đi hợp lệ

Thay vì xét toàn bộ ô trống trên bàn cờ, chương trình chỉ sinh các ô trống lân cận các quân đã đặt trong phạm vi bán kính 2 ô theo mọi hướng. Cách tiếp cận này dựa trên nhận định rằng các nước đi có giá trị chiến thuật thường nằm gần các quân đã có trên bàn.

Trường hợp đặc biệt: nếu bàn cờ chưa có quân nào, nước đi đầu tiên mặc định là ô trung tâm - vị trí có lợi nhất về mặt kiểm soát bàn cờ.

Khi phục vụ thuật toán tìm kiếm, các nước đi ứng viên còn được sắp xếp theo mức độ “tập trung” - ô nào gần nhiều quân hơn được xét trước. Điều này giúp thuật toán Alpha-Beta cắt tỉa hiệu quả hơn vì các nước đi tốt được đánh giá sớm hơn.

5 Cách kiểm tra trạng thái kết thúc

Sau mỗi lượt đánh, chương trình kiểm tra hai điều kiện kết thúc:

Điều kiện thắng: Nếu người chơi hoặc máy tính tạo được 4 quân liên tiếp theo hàng ngang, hàng dọc hoặc đường chéo, bên đó thắng và ván chơi kết thúc.

Điều kiện hòa: Nếu bàn cờ đầy nhưng không có bên nào đạt điều kiện thắng, kết quả là hòa.

6 Thuật toán Minimax

Thuật toán Minimax được áp dụng để tìm nước đi tối ưu bằng cách duyệt toàn bộ cây trạng thái đến độ sâu giới hạn. Nguyên lý cốt lõi là AI (MAX) cố gắng tối đa hóa điểm số, trong khi đối thủ (MIN) cố gắng tối thiểu hóa nó, với giả định cả hai bên đều hành động tối ưu.

1. Ưu tiên tình huống quyết định: Trước khi khởi tạo duyệt cây đệ quy, hệ thống kiểm tra nhanh các nước đi kề cận để xử lý các tình huống hiển nhiên - thắng ngay hoặc buộc phải chặn. Trạng thái này trả về giá trị tuyệt đối, bỏ qua quá trình mở rộng nhánh.

2. Duyệt không gian và điều kiện dừng: Thuật toán dừng đệ quy khi rơi vào một trong hai trường hợp:

- *Trạng thái kết thúc:* Phân định Thắng, Thua hoặc Hòa, trả về trọng số tuyệt đối.
- *Đạt giới hạn độ sâu:* Biến depth bằng 0, hàm đánh giá tĩnh được gọi để ước lượng ưu thế của cấu hình bàn cờ hiện tại.

3. Luân phiên vai trò MAX/MIN: Tại mỗi tầng, nếu là lượt MAX thì chọn nước đi có điểm cao nhất; nếu là lượt MIN thì chọn nước đi có điểm thấp nhất. Hai vai trò luân phiên nhau qua mỗi lần đệ quy.

7 Thuật toán Alpha-Beta

Thuật toán Alpha-Beta được áp dụng nhằm tối ưu hóa quá trình duyệt không gian trạng thái của thuật toán Minimax. Nguyên lý cốt lõi là loại bỏ (cắt tỉa) các nhánh không tiềm năng, giảm thiểu số lượng nút cần đánh giá mà không làm thay đổi kết quả tối ưu cuối cùng.

1. Ưu tiên tình huống quyết định: Trước khi khởi tạo duyệt cây đệ quy, hệ thống đánh giá nhanh các nước đi kề cận để xử lý các tình huống hiển nhiên (thắng hoặc buộc phải chặn). Trạng thái này trả về giá trị tuyệt đối, bỏ qua quá trình mở rộng nhánh.

2. Duyệt không gian và bảo toàn ranh giới: Quá trình đệ quy duy trì hai tham số giới hạn:

- α : Giá trị lớn nhất mà nút MAX đảm bảo đạt được tính đến thời điểm hiện tại.
- β : Giá trị nhỏ nhất mà nút MIN đảm bảo đạt được.

3. Sắp xếp nước đi: Hiệu suất cắt tỉa tỷ lệ thuận với khả năng đánh giá sớm các nước đi tốt. Thuật toán tích hợp bộ sinh nước đi ưu tiên chọn lọc các vị trí tiềm năng (các ô kề lân cận khu vực có quân), giúp nhanh chóng thu hẹp khoảng $[\alpha, \beta]$ và gia tăng số lượng nhánh bị cắt tỉa.

4. Điều kiện dừng và Hàm đánh giá tĩnh: Thuật toán ngắt đệ quy khi rơi vào một trong hai trường hợp:

- *Trạng thái kết thúc:* Phân định Thắng, Thua, hoặc Hòa, trả về trọng số tuyệt đối.
- *Đạt giới hạn độ sâu:* Biến *depth* bằng 0. Hàm đánh giá tĩnh được gọi để ước lượng ưu thế của cấu hình bàn cờ hiện tại.

5. Điều kiện cắt tỉa: Tại bất kỳ nút duyệt nào, nếu ranh giới thỏa mãn điều kiện $\beta \leq \alpha$, thuật toán xác định nhánh hiện tại chắc chắn không được lựa chọn ở các cấp ra quyết định cao hơn. Vòng lặp mở rộng nút bị phá vỡ (break), giúp tiết kiệm tài nguyên bộ nhớ và thời gian xử lý.

8 Hàm đánh giá trạng thái

8.1 Tổng quan phương pháp

Trong cấu trúc thuật toán tìm kiếm đối kháng, hàm đánh giá heuristic định lượng giá trị của một trạng thái bàn cờ tại độ sâu giới hạn ($depth = 0$). Tổng điểm đánh giá ($Score_{total}$) được xác định bằng hiệu số giữa tổng điểm cấu trúc của AI và điểm cấu trúc của đối thủ:

$$Score_{total} = Score_{AI} - Score_{Player} \quad (1)$$

Theo hệ quy chiếu này:

- $Score_{total} > 0$: Trạng thái mang lại lợi thế cho AI.
- $Score_{total} < 0$: Trạng thái mang lại lợi thế cho đối thủ.
- $Score_{total} = 0$: Trạng thái cân bằng.

8.2 Thuật toán quét và trích xuất đặc trưng

Điểm số được tính toán dựa trên việc phân tích các chuỗi quân cờ đồng chất trên không gian lưới $N \times N$. Sức mạnh của một chuỗi được xác định bởi hai đặc trưng cơ sở: độ dài chuỗi (L) và số lượng đầu mở (E - đại diện cho bậc tự do).

Thuật toán duyệt toàn bộ không gian và thực hiện các bước sau tại tọa độ (x, y) chứa quân cờ cần xét:

1. **Phân tích vector hướng:** Khảo sát sự tồn tại của chuỗi theo 4 vector hướng độc lập trong không gian 2D: dọc $(1, 0)$, ngang $(0, 1)$, chéo chính $(1, 1)$, và chéo phụ $(1, -1)$.
2. **Tối ưu hóa đếm (Kiểm soát trùng lặp):** Đảm bảo độ phức tạp tuyến tính bằng cách chỉ đánh giá chuỗi tại phần tử đầu tiên. Kiểm tra tọa độ liền trước $(x - dx, y - dy)$ theo vector ngược hướng; nếu chứa quân đồng chất, ô (x, y) bị bỏ qua.
3. **Định lượng đặc trưng chuỗi:**
 - *Độ dài (L):* Đếm số lượng phần tử đồng chất liên tiếp theo vector hướng.
 - *Bậc tự do (E):* Kiểm tra số lượng ô trống kề tại hai đầu mút ($E \in \{0, 1, 2\}$). Nếu $E = 0$ (chuỗi bị chặn toàn phần), chuỗi không có giá trị phát triển chiến thuật và nhận trọng số 0.

8.3 Hệ thống trọng số

Sau khi trích xuất cặp đặc trưng (L, E) , thuật toán sử dụng bảng tra cứu để ánh xạ thành điểm số. Hệ thống trọng số được thiết kế phân cấp lũy thừa nhằm ưu tiên tuyệt đối các trạng thái đạt điều kiện thắng hoặc các đe dọa tất yếu.

Độ dài (L)	Đầu mở (E)	Trọng số	Đánh giá chiến thuật
4	1 hoặc 2	100,000	Trạng thái thắng quyết định.
3	2	50,000	Đe dọa bậc 1 (Không thể phòng thủ toàn phần bằng 1 nước).
3	1	1,000	Đe dọa bậc 2 (Yêu cầu hành động phòng thủ tức thời).
2	2	500	Cấu trúc nền tảng chiến thuật.
2	1	100	Cấu trúc thứ cấp.
1	2	10	Đơn vị cờ độc lập có không gian mở.
1	1	1	Đơn vị cờ độc lập bị giới hạn một phần.

Bảng 2: Bảng phân bổ trọng số đánh giá cấu trúc chuỗi quân cờ.

Tổng điểm của một tác nhân là tổng tuyến tính các trọng số từ các chuỗi hợp lệ. Sự chênh lệch theo hàm số mũ giữa các mức trọng số đảm bảo thuật toán hội tụ vào việc ưu tiên giải quyết các cấu trúc chí mạng thay vì tích lũy cấu trúc thứ cấp.

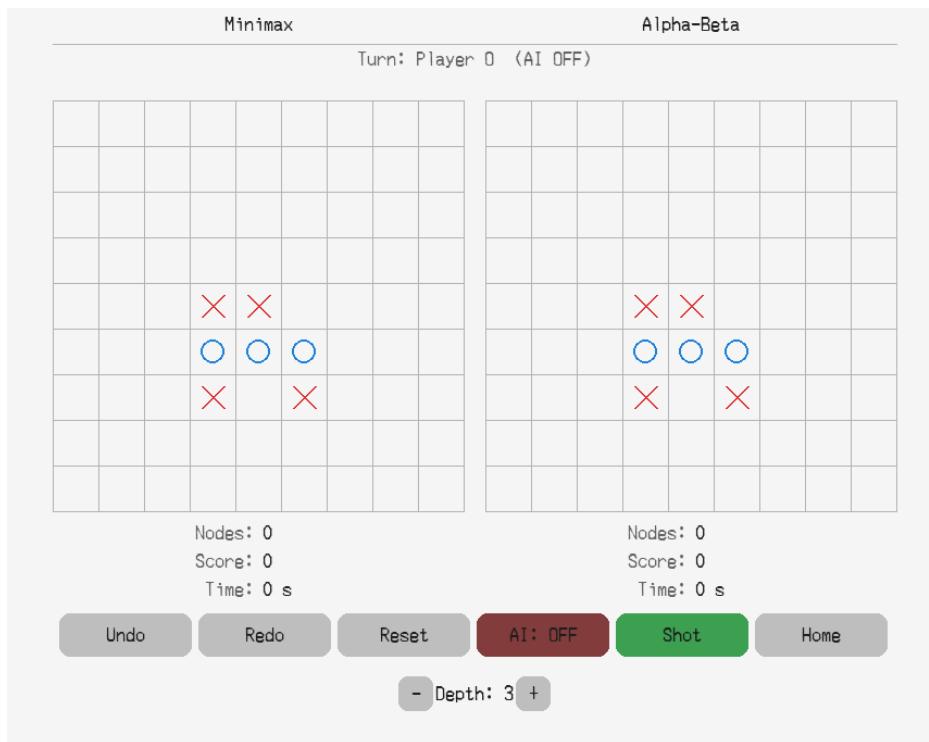
9 Thiết kế các trạng thái thử nghiệm

9.1 Thiết kế các trạng thái điển hình

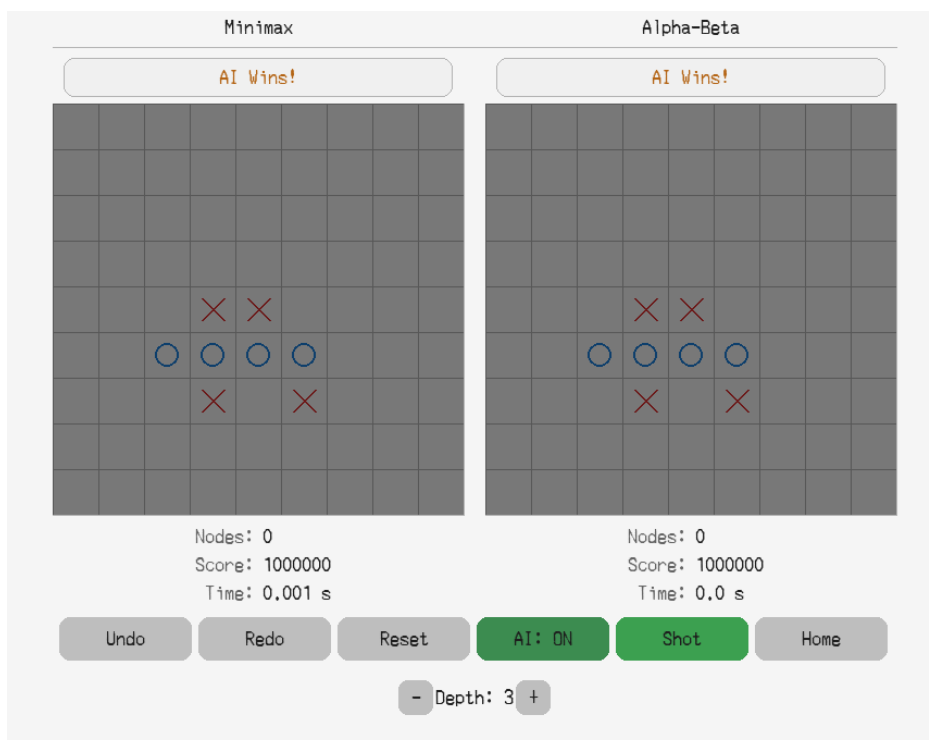
Năm trạng thái thử nghiệm được thiết kế trên bàn cờ Caro 9×9 , bao gồm các tình huống điển hình từ giai đoạn đầu đến kết thúc ván đấu. Mỗi trạng thái được chạy song song trên cả hai thuật toán Minimax và Alpha-Beta Pruning với độ sâu tìm kiếm $d = 3$.

Trạng thái 1 – AI thắng ngay

AI (O) nhận diện được nước thắng của mình và thực hiện ngay lập tức, kết thúc ván với $Score = 1,000,000$.



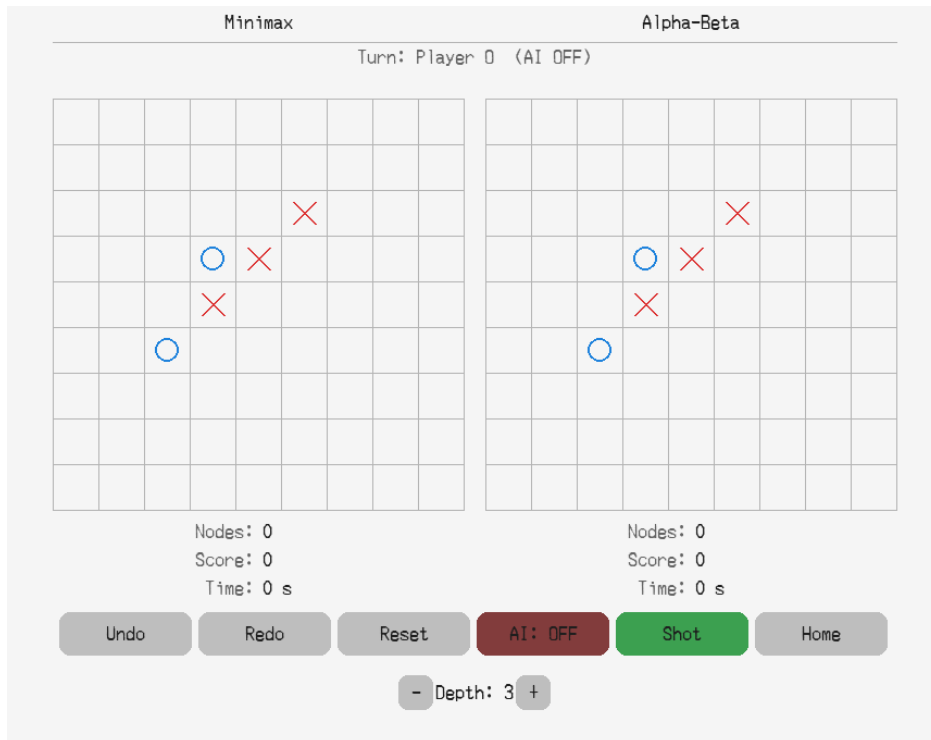
Trạng thái 1 – Trước khi AI đánh



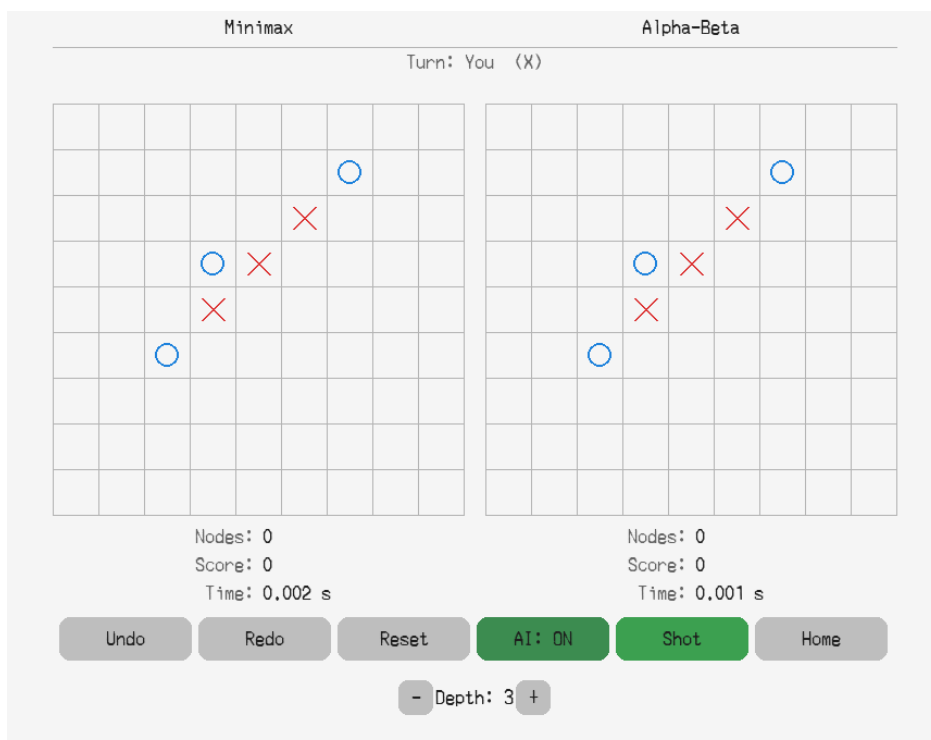
Trạng thái 1 – Sau khi AI đánh

Trạng thái 2 – AI phải chặn

X đang có chuỗi 3 liên tiếp đe dọa thắng, AI buộc phải ưu tiên chặn thay vì tấn công.



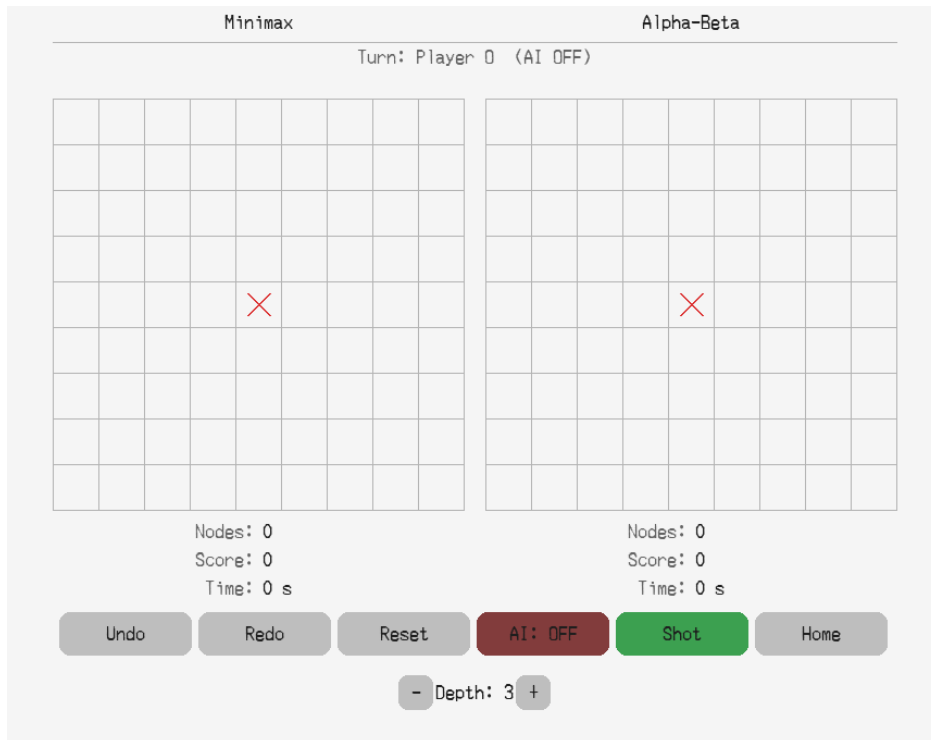
Trạng thái 2 – Trước khi AI đánh



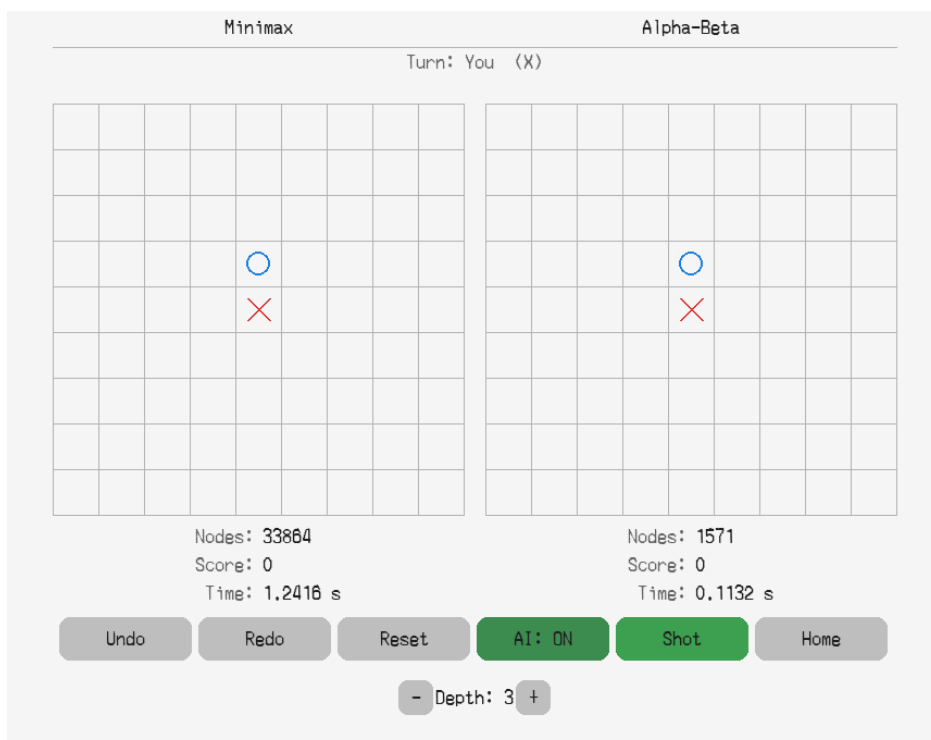
Trạng thái 2 – Sau khi AI đánh

Trạng thái 3 – Đầu ván, ít quân

Bàn cờ chỉ có vài quân, không gian tìm kiếm rộng nhất. AI xác định nước đi chiến lược mở đầu.



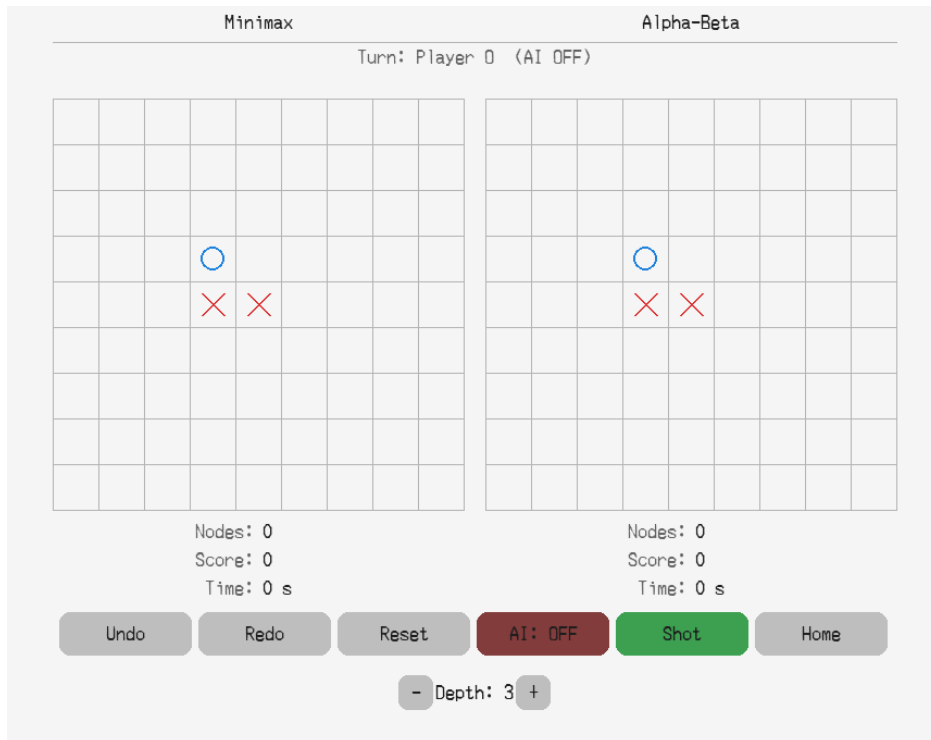
Trạng thái 3 – Trước khi AI đánh



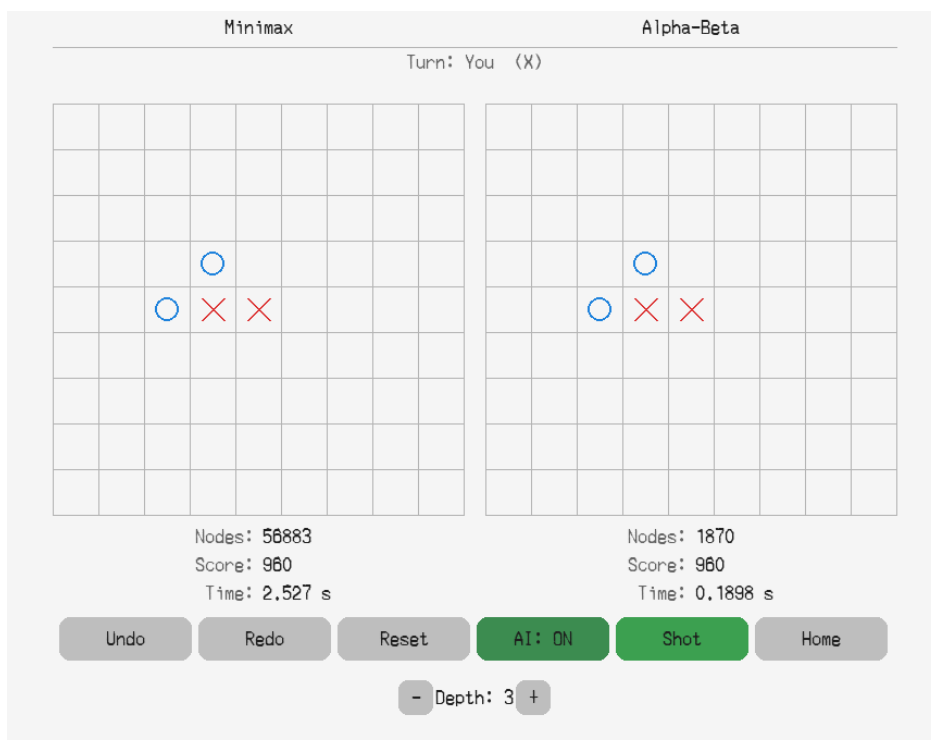
Trạng thái 3 – Sau khi AI đánh

Trạng thái 4 – AI cần chặn một đầu

X triển khai chuỗi 2 liên tiếp, O cần chặn 1 đầu để ngăn cản X, ưu tiên chặn phía bên trái để vừa phòng thủ vừa tấn công.



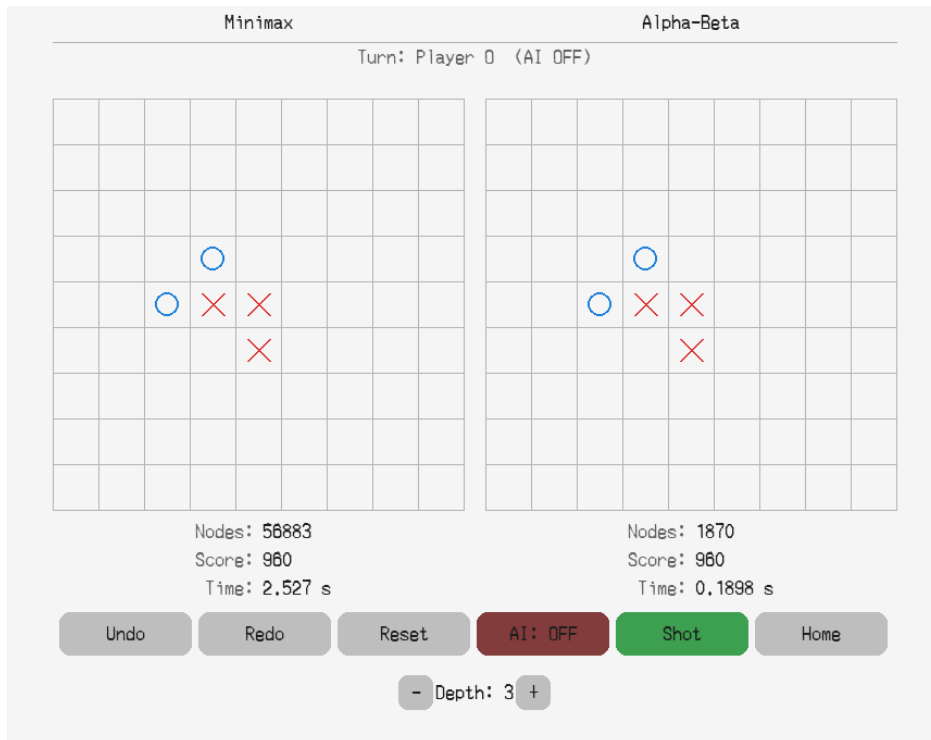
Trạng thái 4 – Trước khi AI đánh



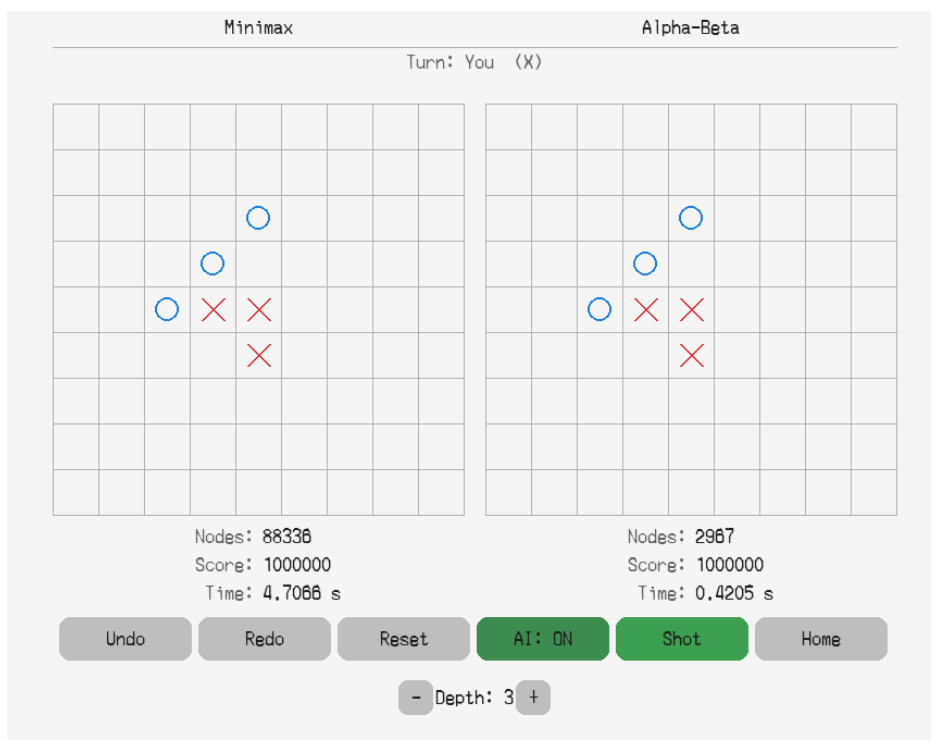
Trạng thái 4 – Sau khi AI đánh

Trạng thái 5 – AI chặn hoặc triển khai 3 liên tiếp

X có 3 quân đang triển khai, AI cần quyết định giữa chặn ngay hoặc triển khai chuỗi của mình.



Trạng thái 5 – Trước khi AI đánh



Trạng thái 5 – Sau khi AI đánh

9.2 Thiết kế các trạng thái với các độ sâu khác nhau

Ba trạng thái thử nghiệm được thiết kế trên bàn cờ Caro 9×9 . Mỗi trạng thái được chạy song song trên cả hai thuật toán Minimax và Alpha-Beta Pruning với các độ sâu tìm kiếm khác nhau.

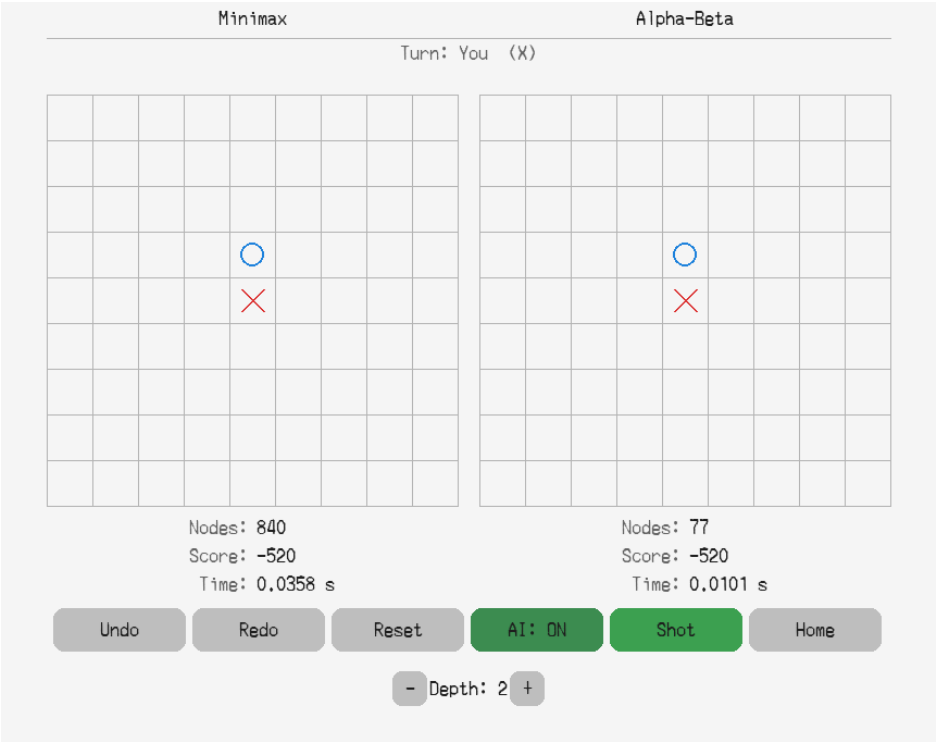
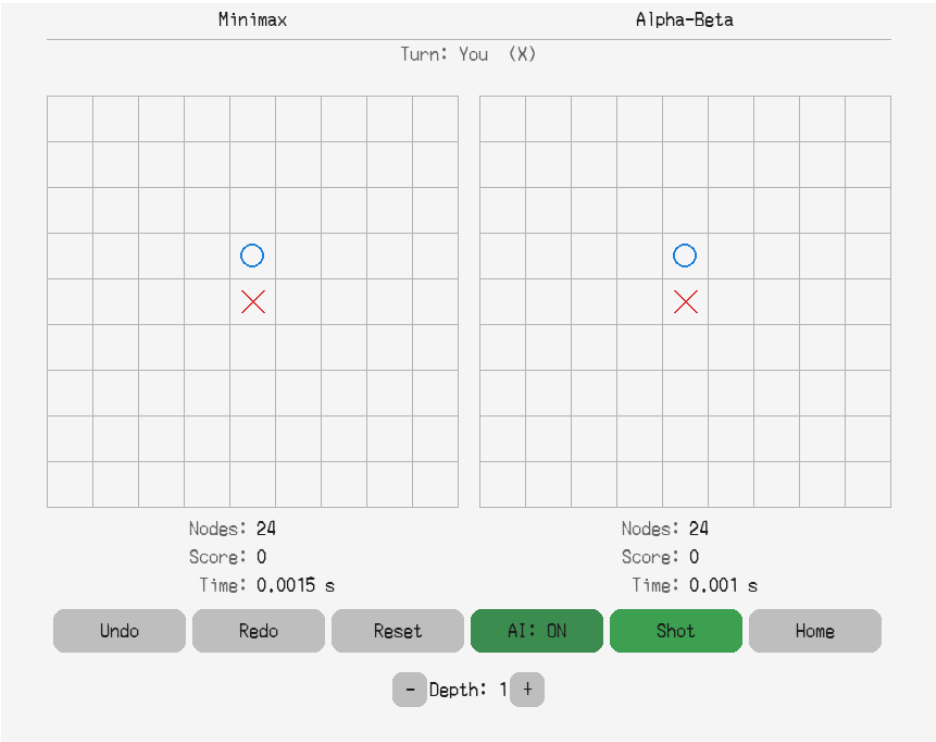
Trạng thái 1 – Khai cuộc, người chơi đánh trước

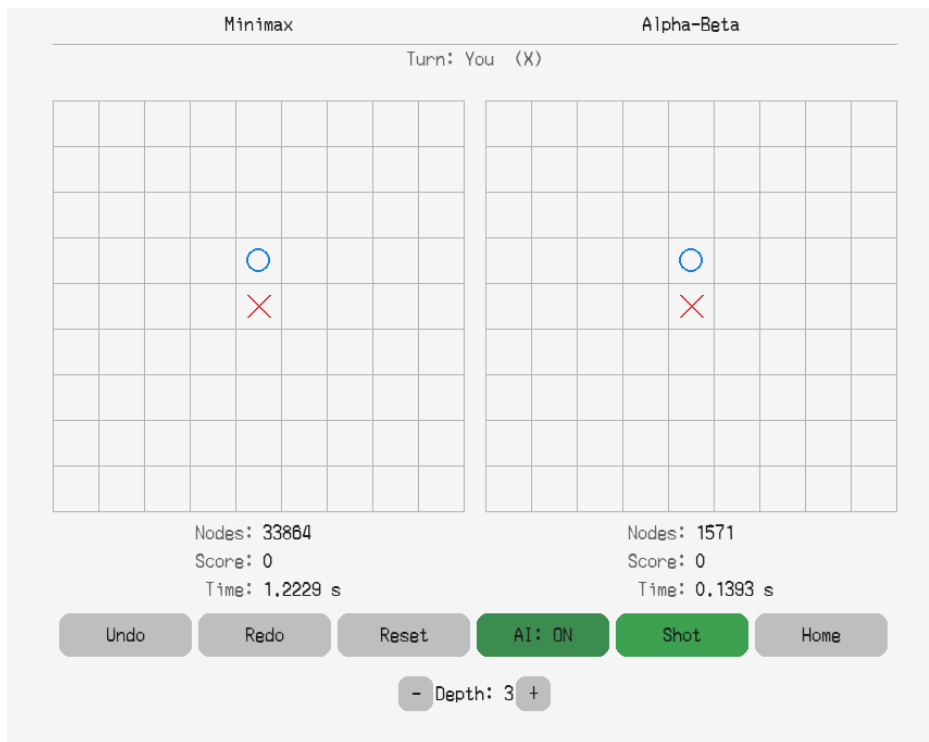
Bàn cờ ở trạng thái khởi đầu, người chơi (X) vừa đặt quân đầu tiên vào trung tâm bàn cờ.



Trạng thái 1 – Khai cuộc, X đánh vào trung tâm

Trạng thái 1 – Kết quả sau khi AI đánh với các độ sâu khác nhau

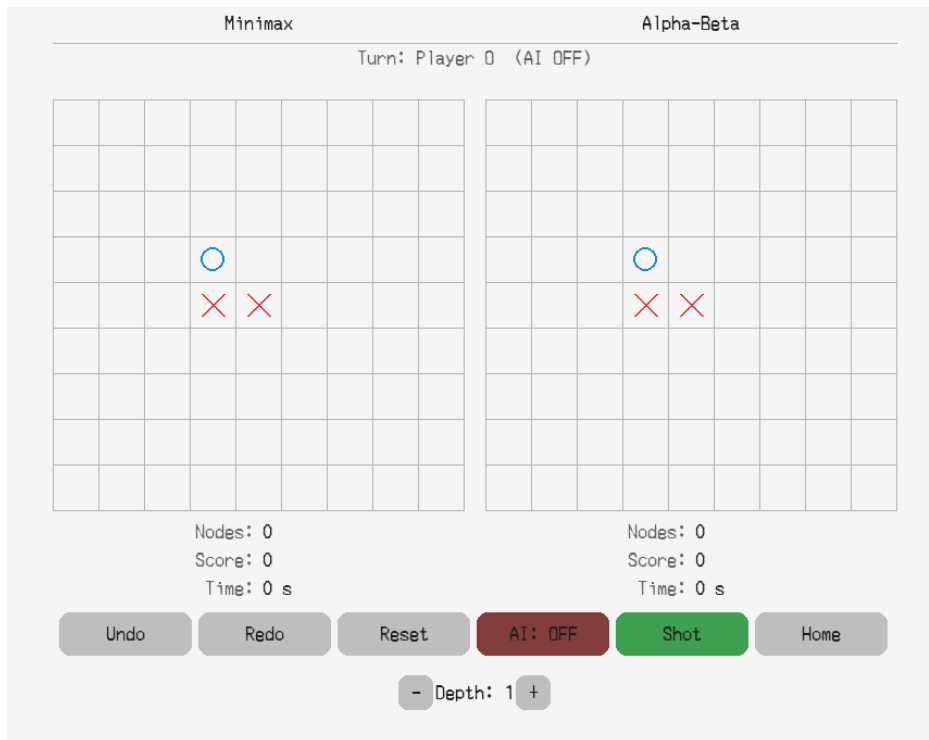




Hình so sánh kết quả AI sau khi đánh ở trạng thái 1 với độ sâu tìm kiếm khác nhau.

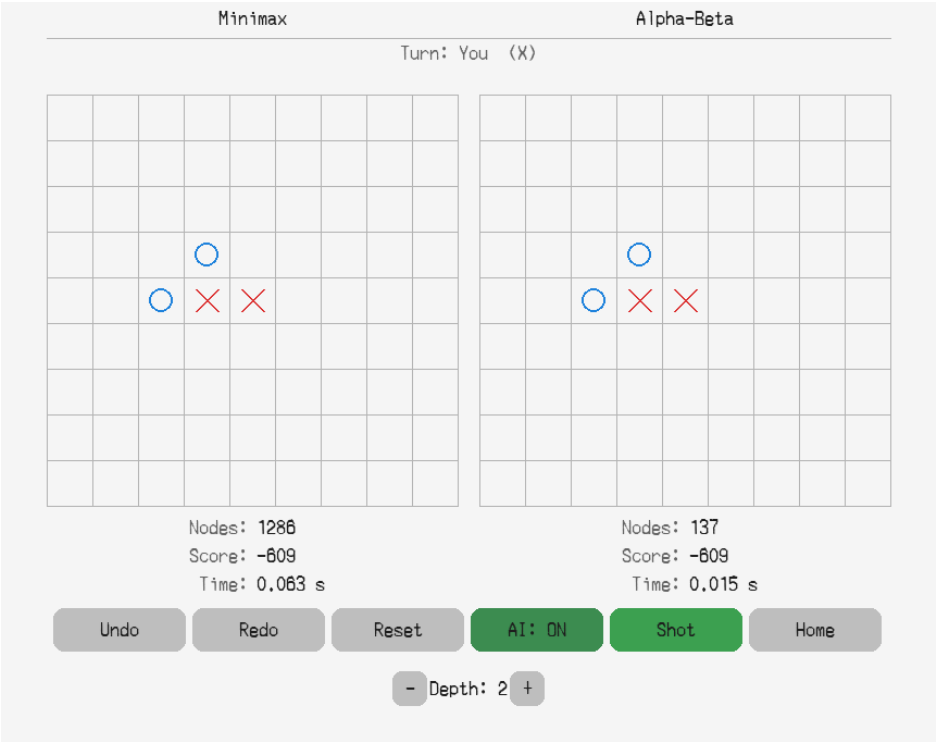
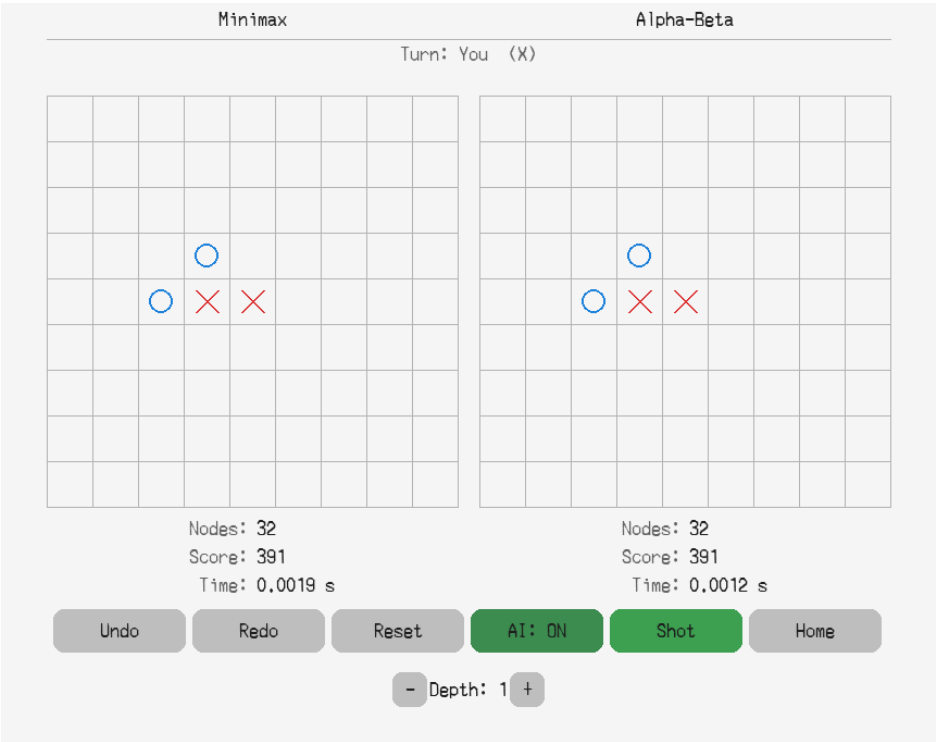
Trạng thái 2 – AI phản công sau nước đi của người chơi

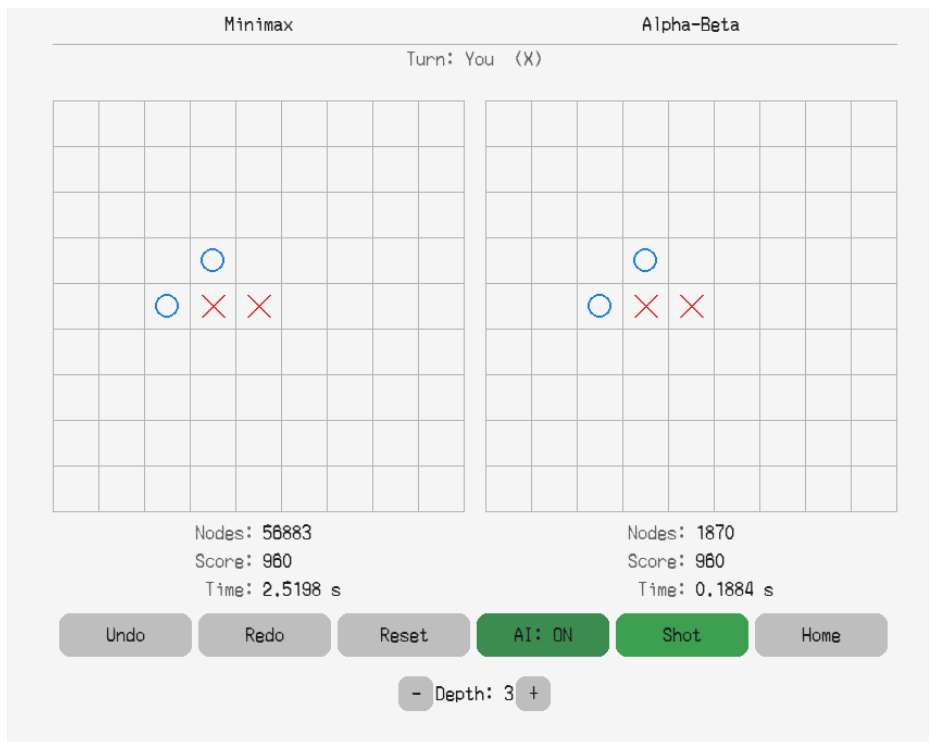
Người chơi (X) có hai quân liên tiếp theo chiều ngang, AI (O) đã đặt một quân phía trên nhằm kiểm soát trung tâm và ngăn X mở rộng chuỗi.



Trạng thái 2 – AI phản công

Trạng thái 2 – Kết quả sau khi AI đánh với các độ sâu khác nhau

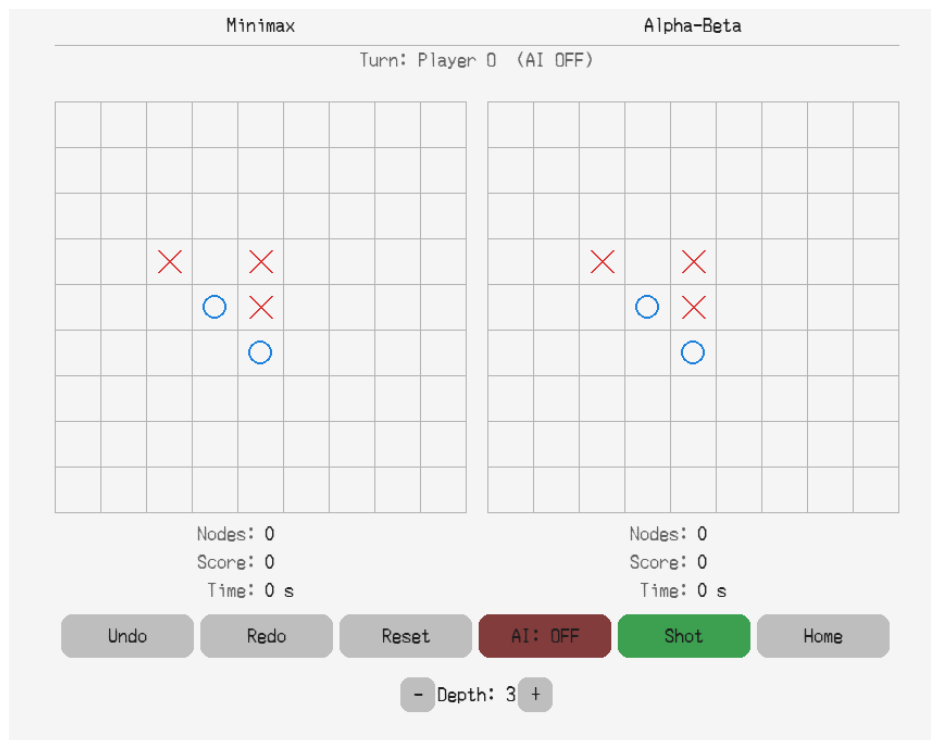




Hình so sánh kết quả AI sau khi đánh ở trạng thái 2 với độ sâu tìm kiếm khác nhau.

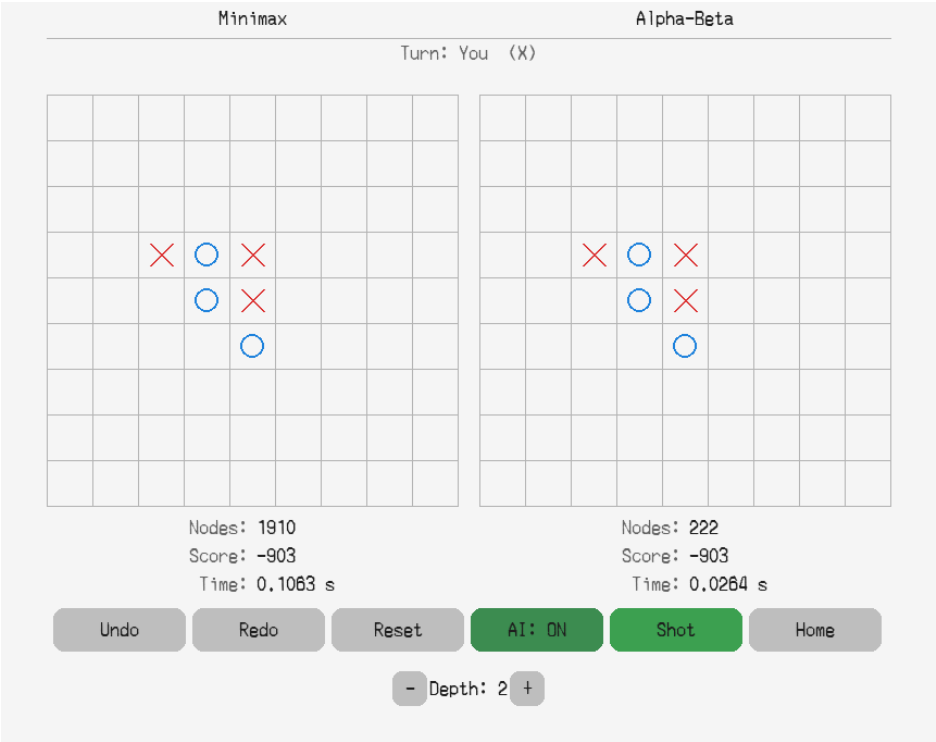
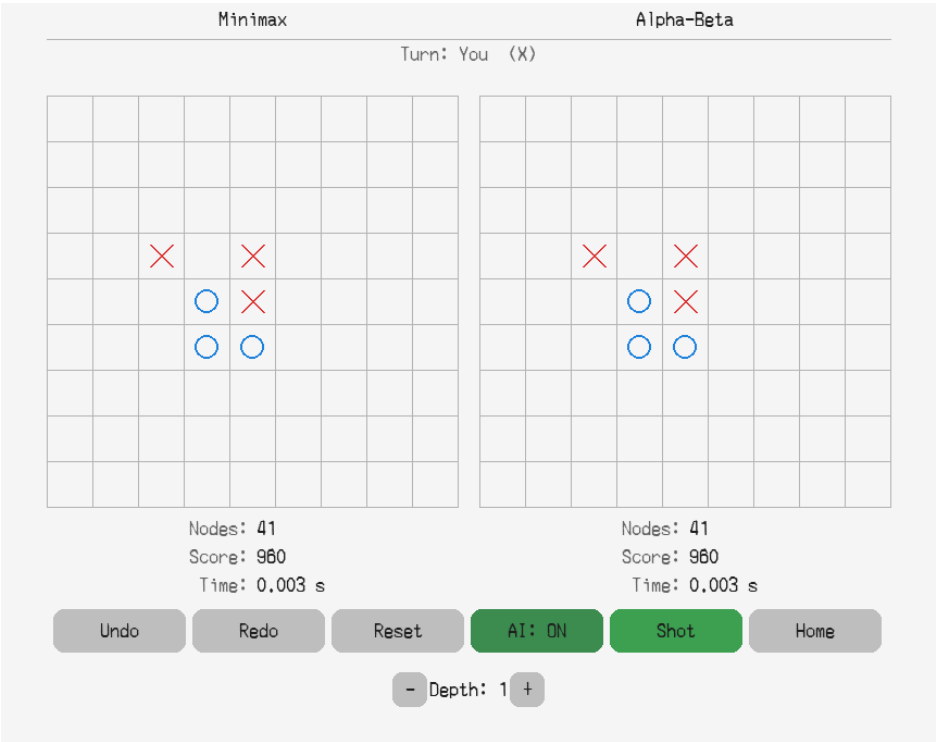
Trạng thái 3 – AI triển khai tấn công

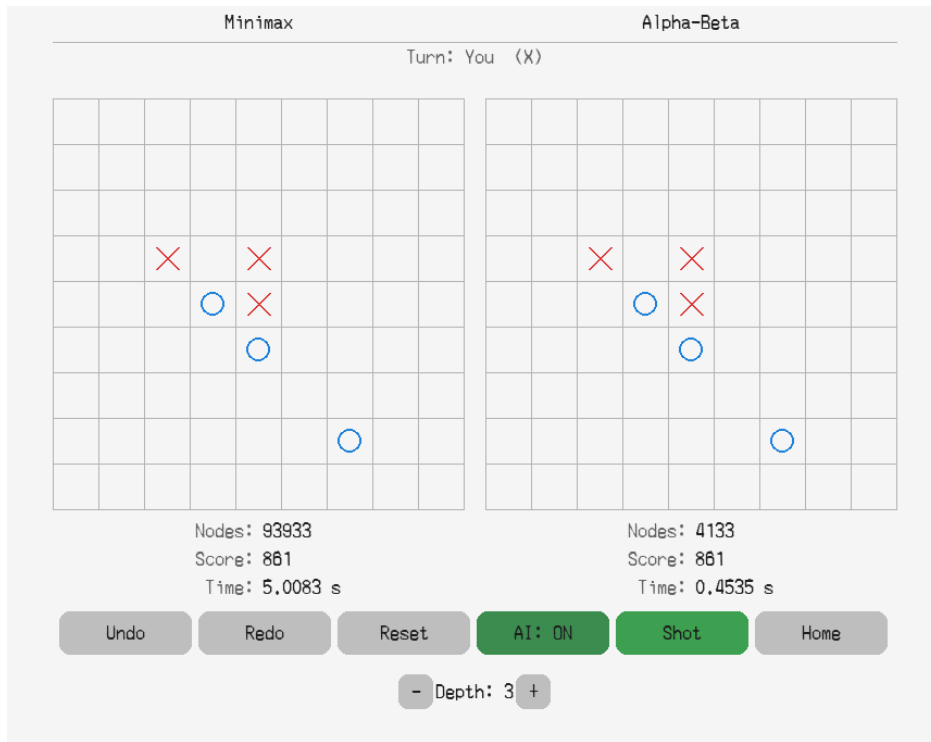
Người chơi (X) vừa chặn 1 đầu chuỗi 2 liên tiếp của (O) và đe dọa tạo thành chuỗi 3 liên tiếp.



Trạng thái 3 – AI triển khai tấn công

Trạng thái 3 – Kết quả sau khi AI đánh với các độ sâu khác nhau





Hình so sánh kết quả AI sau khi đánh ở trạng thái 3 với độ sâu tìm kiếm khác nhau.

10 Bảng kết quả thực nghiệm

TT	Mô tả	Minimax			Alpha-Beta Pruning		
		Nodes	Score	Time (s)	Nodes	Score	Time (s)
1	AI thắng ngay	0	1 000 000	0.001	0	1 000 000	0.000
2	AI phải chặn	0	0	0.002	0	0	0.001
3	Đầu ván, ít quân	33 864	0	1.2416	1 571	0	0.1132
4	AI cần chặn một đầu	56 883	960	2.5270	1 870	960	0.1898
5	AI chặn hoặc triển khai	88 336	1 000 000	4.7066	2 967	1 000 000	0.4205
Trung bình		59 694		2.8251	2 136		0.2412

Bảng 2: Kết quả thực nghiệm so sánh Minimax và Alpha-Beta Pruning ($d = 3$, bàn 9×9)

Trạng thái	Depth	Minimax			Alpha-Beta Pruning		
		Nodes	Score	Time (s)	Nodes	Score	Time (s)
1	1	24	0	0,0015	24	0	0,001
1	2	840	-520	0,0358	77	-520	0,0101
1	3	33 864	0	1,2229	1 571	0	0,1393
2	1	32	391	0,0019	32	391	0,0012
2	2	1 286	-609	0,063	137	-609	0,015
2	3	56 883	960	2,5198	1 870	960	0,1884
3	1	41	960	0,003	41	960	0,003
3	2	1 910	-903	0,1063	222	-903	0,0264
3	3	93 933	861	5,0083	4 133	861	0,4535

Bảng 3: So sánh Minimax và Alpha-Beta Pruning theo các độ sâu tìm kiếm.

11 Nhận xét về số trạng thái đã xét và thời gian chạy

Từ kết quả thực nghiệm ở Bảng 10, có thể rút ra các nhận xét sau:

Về số nút đã xét: Với các trạng thái thực sự duyệt cây (trạng thái 3–5), Alpha-Beta Pruning chỉ xét trung bình 2,136 nút so với 59,694 nút của Minimax, tức cắt giảm được 96,4% số nút cần duyệt. Đối với trạng thái 1 và trạng thái 2, cả hai thuật toán đều trả về ngay lập tức với Nodes = 0 do cơ chế ưu tiên tình huống quyết định: AI nhận diện nước thắng hoặc nước chặn bắt buộc trước khi khởi động đệ quy, nên không cần duyệt cây.

Về thời gian chạy: Xét trên các trạng thái có duyệt cây (trạng thái 3–5), Alpha-Beta nhanh hơn Minimax trung bình 11,7 lần (0.241 s so với 2.825 s). Khoảng cách lớn nhất xuất hiện ở trạng thái 5 – trạng thái phức tạp nhất với $Score = 10^6$ – khi Minimax mất 4.707 s trong khi Alpha-Beta chỉ cần 0.421 s. Ở trạng thái 1

và trạng thái 2, cả hai thuật toán đều phản hồi tức thì (dưới 0.003 s) nhờ cơ chế phát hiện sớm tình huống hiển nhiên.

Về chất lượng quyết định: Cả hai thuật toán cho cùng kết quả Score ở mọi trạng thái thử nghiệm, chứng tỏ Alpha-Beta không làm thay đổi nước đi được chọn so với Minimax, chỉ loại bỏ các nhánh không cần thiết.

Kết luận: Alpha-Beta Pruning vượt trội hoàn toàn so với Minimax về tốc độ trong khi vẫn đảm bảo chất lượng nước đi. Với $d = 3$ trên bàn 9×9 , Alpha-Beta đáp ứng tốt yêu cầu thời gian thực (dưới 0.5 s ở mọi trạng thái), còn Minimax có thể mất gần 5 s – không khả thi khi tăng độ sâu tìm kiếm.

12 Nhận xét về ảnh hưởng của độ sâu tìm kiếm

12.1 Ảnh hưởng đến thời gian chạy

Thời gian chạy tăng đột biến theo độ sâu ở cả hai thuật toán. Với Minimax, từ độ sâu 1 lên độ sâu 3, số nút cần xét tăng hàng chục nghìn lần (trạng thái 1: từ 24 lên 33.864 nút; trạng thái 3: từ 41 lên 93.933 nút). Alpha-Beta kiểm soát tốt hơn nhờ cắt tỉa - cùng điều kiện, số nút chỉ tăng từ vài chục lên vài nghìn (trạng thái 1: từ 24 lên 1.571; trạng thái 3: từ 41 lên 4.133). Ở độ sâu 3, Minimax chậm hơn Alpha-Beta trung bình **10–13 lần** trên cùng trạng thái.

12.2 Ảnh hưởng đến kết quả nước đi

Với **trạng thái 1 và 2**, mặc dù score thay đổi theo từng mức độ sâu, cả hai thuật toán đều chọn **cùng một nước đi** ở mọi độ sâu. Điều này cho thấy cấu trúc của hai trạng thái này tương đối rõ ràng - dù tầm nhìn ngắn hay dài, nước đi tốt nhất vẫn là một.

Với **trạng thái 3**, kết quả khác biệt rõ rệt hơn: không chỉ score thay đổi (960, -903, 861 ở độ sâu 1, 2, 3) mà **nước đi được chọn cũng khác nhau** giữa các mức độ sâu. Ở độ sâu 1, AI chỉ thấy lợi ích trước mắt và đặt quân gần cụm

hiện tại; ở độ sâu 3, AI nhìn xa hơn và chọn vị trí hoàn toàn khác để cân bằng giữa tấn công và phòng thủ. Đây là minh chứng rõ nhất cho thấy **độ sâu tìm kiếm ảnh hưởng trực tiếp đến chất lượng quyết định** trong các tình huống phức tạp, khi bàn cờ có nhiều mối đe dọa đan xen.

13 Ưu điểm và hạn chế của chương trình

13.1 Ưu điểm

Về thuật toán:

- Alpha-Beta Pruning cắt tỉa hiệu quả, giảm đáng kể số nút cần xét so với Minimax thuần túy ở cùng độ sâu — trung bình 10–13 lần trên các trạng thái thực nghiệm.
- Cơ chế kiểm tra mối đe dọa trước khi tìm kiếm giúp xử lý ngay các tình huống thắng hoặc chặn hiển nhiên mà không tốn thêm chi phí duyệt cây.
- Bộ sinh nước đi chỉ xét vùng lân cận quân đã đặt, tránh bùng nổ không gian tìm kiếm. Việc sắp xếp nước đi theo mức độ tập trung quân giúp Alpha-Beta cắt tỉa sâu hơn ở các nhánh đầu.

Về chương trình:

- Hỗ trợ cả hai chế độ: hai người chơi và người đấu với AI.
- Có chức năng hoàn tác và làm lại nước đi, chụp ảnh trạng thái bàn cờ, thuận tiện cho việc phân tích ván đấu.
- Cho phép so sánh trực tiếp hiệu năng giữa Minimax và Alpha-Beta qua màn hình thực nghiệm với các thông số nodes, score và thời gian chạy.

13.2 Hạn chế

Về thuật toán:

- Độ sâu tìm kiếm bị giới hạn do chi phí tính toán, khiến AI có thể bỏ lỡ các nước đi chiến lược dài hạn vượt ngoài tầm nhìn.
- Hàm đánh giá heuristic chỉ dựa trên chuỗi quân liên tiếp, chưa xét các mẫu phức tạp như đòn kép hay bày nhiều hướng.
- Bán kính sinh nước đi cố định (2 ô) có thể bỏ sót một số nước đi có giá trị ở xa vùng đang thi đấu.

14 Tài liệu tham khảo

Trong quá trình thực hiện đề tài, nhóm có tham khảo 2 repo mẫu do thầy cung cấp để học hỏi cách tổ chức thư mục, kiến trúc dự án và phân chia các module chức năng. Dựa trên đó, nhóm tự nghiên cứu, cài đặt và hoàn thiện các phần logic trò chơi, thuật toán AI và thực nghiệm.